1. Overview

GOFR, the Grocery Operations and Fulfillment Robot, is a full-stack robotic grocery assistant designed to support order fulfillment and store-side item handling. It combines a mecanum-drive mobile base, LiDAR-based navigation, a ViperX/VX300S robotic arm, RealSense vision, local ultrasonic sensing, ROS 2 task orchestration, a Node.js/PostgreSQL backend, and browser-based user interfaces.

The intent of this document is to bring a new user, store operator, or maintainer up to speed so that they can install, start, operate, stop, and recover the product. The installation section assumes that the robot may be moved to another indoor test area or store-like aisle. The User's Manual assumes that setup has already been completed and focuses on safe operation by customer, store employee, and maintainer roles.

This document should be reviewed during customer handoff. The installer should walk the client through the physical stop controls, Fleet Manager pages, customer ordering flow, semantic map editing workflow, and maintainer diagnostic checks before allowing normal operation.

2. System Overview

Author source: Pree for the high-level system description and interfaces; Darren for the physical chassis description; Bach for the custom robotic-arm prototype description; Lion for final consolidation and technical expansion.

2.1 Major Subsystems

GOFR is a full-stack robotic system with the following major subsystems:

- **Mobile base:** mecanum-drive chassis controlled by an STM32 motor-control board.
- **Navigation stack:** ROS 2 Humble, Cartographer, EKF-based odometry fusion, and Nav2.
- **Manipulation stack:** Interbotix ViperX/VX300S arm with MoveIt 2 and a ROS action server.
- **Vision stack:** Intel RealSense camera with YOLO-based detection and target pose publication.
- **Safety sensing:** ESP32-based ultrasonic sensor nodes publishing four-direction collision alerts.
- **Backend and UI:** Node.js + PostgreSQL backend with a customer web app and employee fleet dashboard using the Vue.js framework.
- **Task orchestration:** ROS 2 Behavior Trees that connect orders, semantic targets, navigation, vision, and arm control.

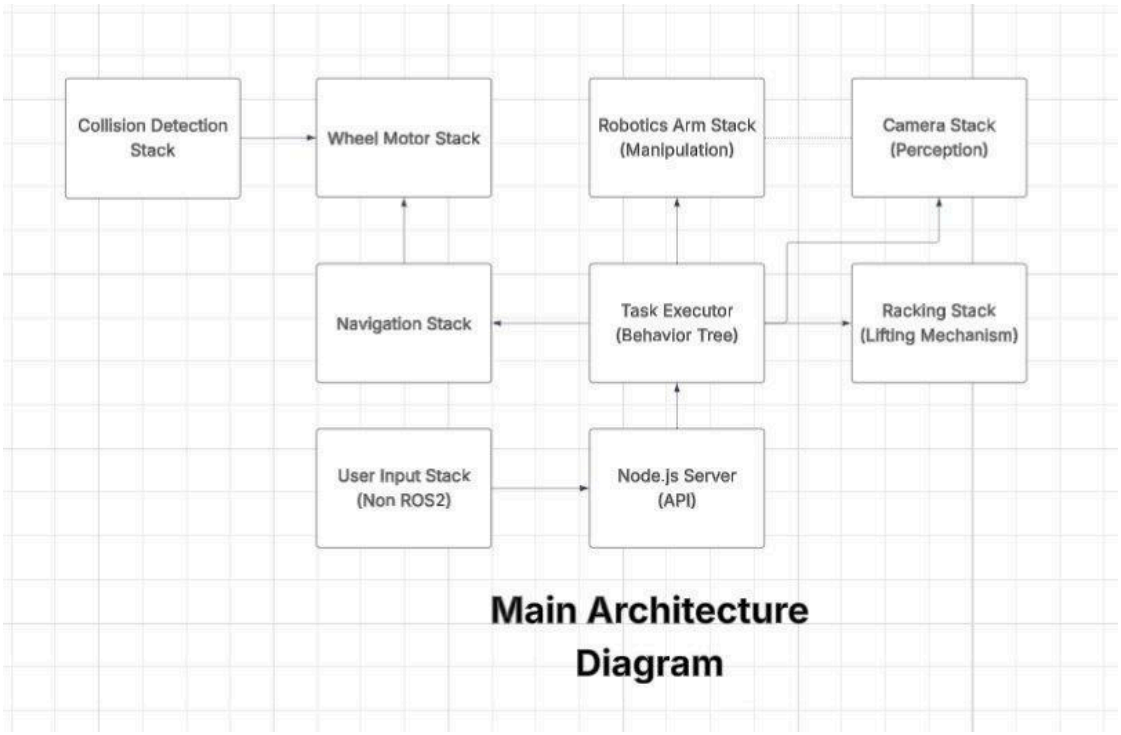


Figure 2. Main system architecture diagram retained from the team draft.

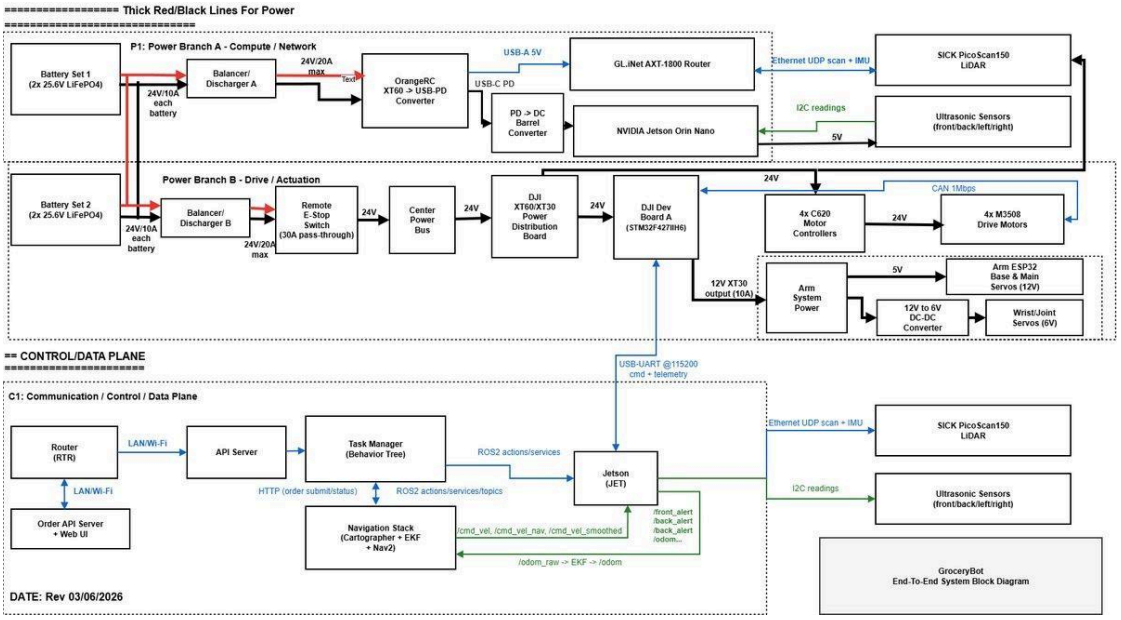


Figure 3. Electrical and communication diagram retained from the team draft.

2.2 High-Level Capabilities

- Accept customer shopping orders and employee restock-order requests from the web application.
- Maintain customer membership, inventory data, employee login, and semantic-map data in PostgreSQL.
- Localize the robot in a mapped store environment using LiDAR, IMU, and odometry.

- Dynamically adjust product locations on the map using the Fleet Manager Store Map workflow.
- Navigate to a selected aisle, rack, or product slot location.
- Detect and verify a target object using RealSense and YOLO.
- Command the ViperX arm to pick and place an item.
- Show store-map and SLAM-related views in the employee-facing interface.
- Provide AI-assisted and multilingual support in both applications when the Gemini API path is configured.

2.3 User-Facing Interfaces

Interface	Primary user	Purpose
Customer web interface	Customer or demo user	Browser-based shopping interface served by the backend. It supports member login or guest flow, inventory browsing, cart building, order submission, multilingual page variants, and an optional AI assistant panel.
Employee Fleet Manager dashboard	Store employee or operator	Vue-based dashboard for inventory reports, semantic Store Map editing, SLAM viewing, robot status, restock submission, employee accounts, and AI-assisted interaction.
JetKVM remote console	Maintainer or integration operator	Remote console for the JetPack / robot host. Used for software installation, ROS commands, mapping, diagnostics, and recovery.

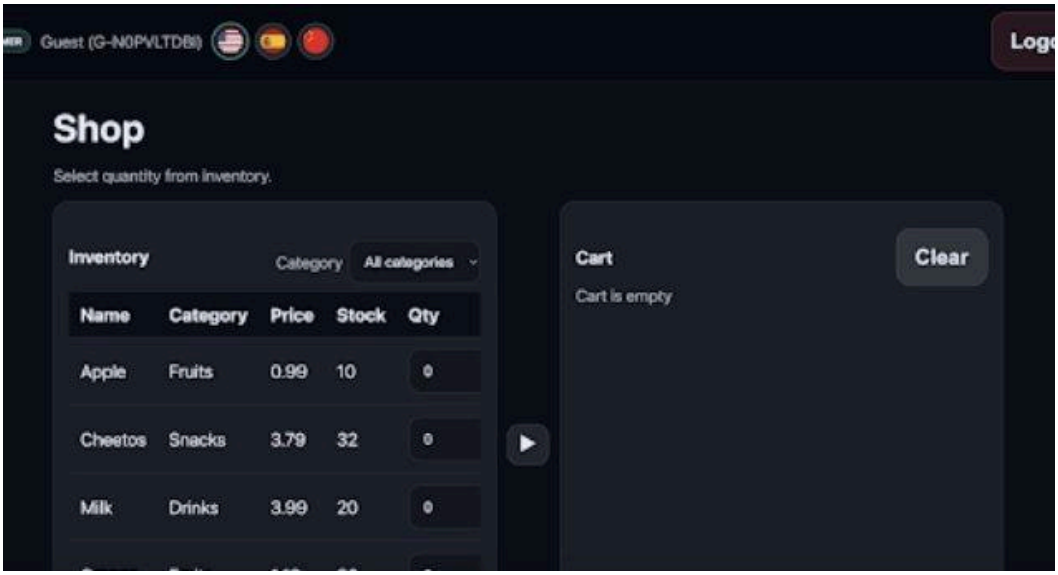


Figure 4. Customer shopping interface example retained from the team draft.

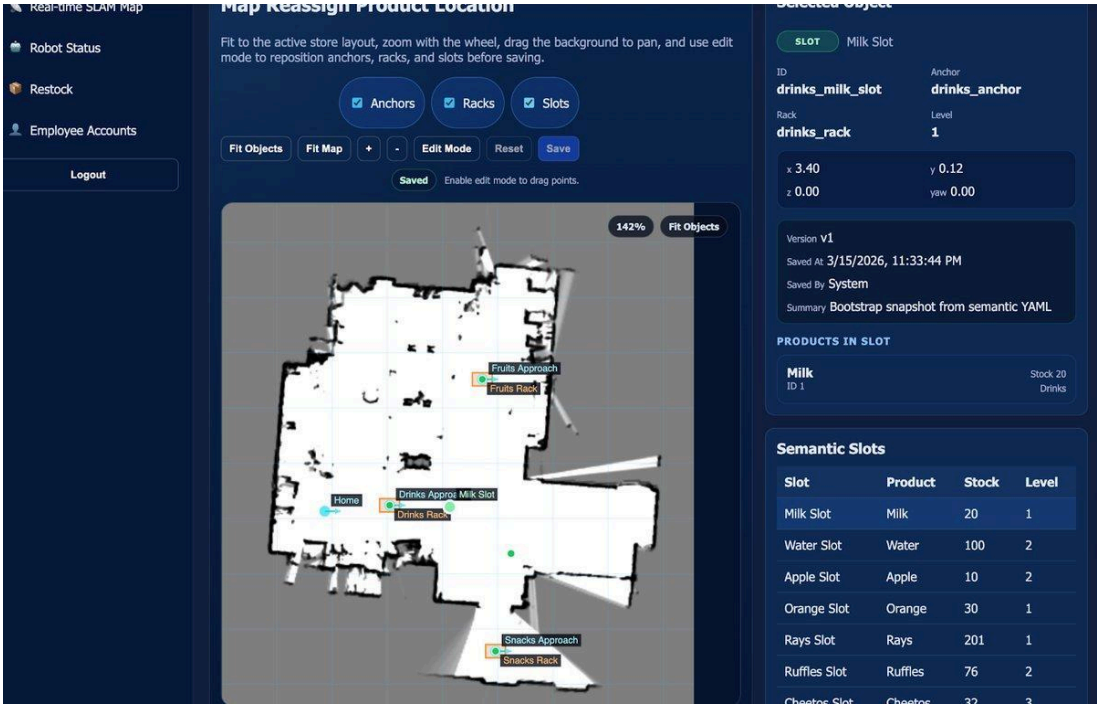


Figure 5. Fleet Manager / map-related interface example retained from the team draft.

2.4 Physical Description of the Robot

Darren: The 20 inch by 20 inch chassis is made of 10-series 8020 and includes two main layers. The first layer houses most of the electrical components of the robot, including the LiDAR, motors, batteries, and Jetson-class robot computer. Four 6 inch mecanum wheels are used. These components are placed into 3D-printed housings and mounted on a 0.125 inch by 14 inch by 14 inch aluminum plate. The second layer is elevated 10 inches above the first layer and includes a GeekPi 9 inch LCD monitor, ViperX 300S robotic arm, and basket for holding groceries.

Darren: Acrylic plates are used as the outer casing along with 3D-printed parts that slide into the 8020 bars. The slots in these printed parts allow users to slide the acrylic plates in and out for easier assembly and disassembly.

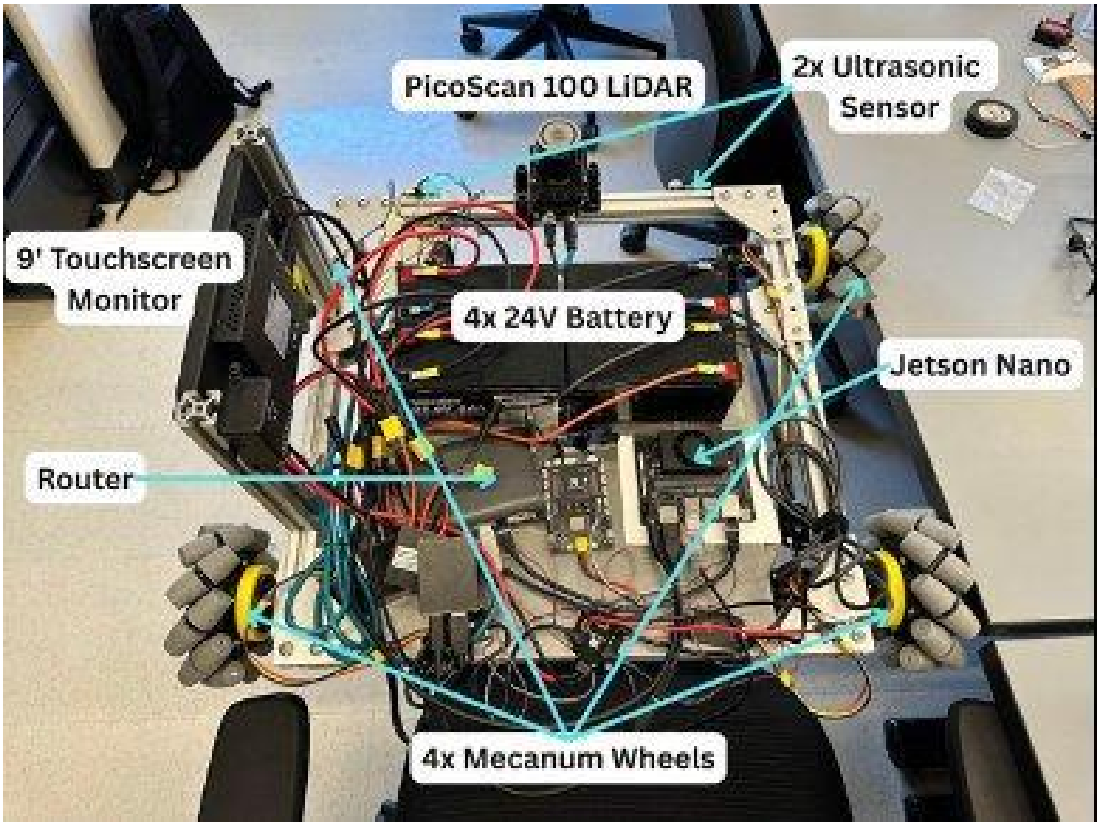


Figure 6. Annotated physical components from the team draft.

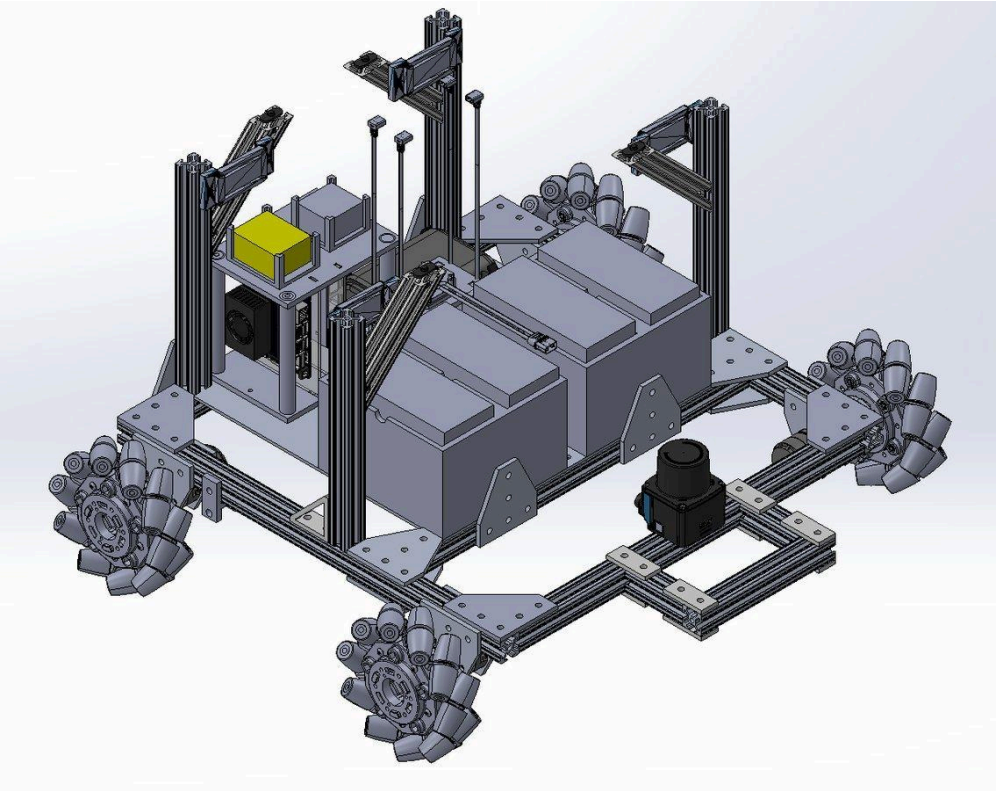


Figure 7. CAD view of the GOFR chassis.

2.5 Custom Robotic Arm Prototype

Bach: The custom robotic arm is built using 3D-printed components, 8020 linkages, a turntable bearing, and servo motors. Major components such as the base and gripper were 3D printed because this fabrication process is fast and highly customizable. The 8020 linkages improve modularity because arm reach can be adjusted by changing linkage length. The 8020 built-in connection hubs also simplify assembly by reducing the need for drilling or designing extra connection layers. To maximize strength-to-weight ratio and minimize weight, multiple servo motors with different torque ratings were selected.

Prototype boundary: The custom arm is documented for team continuity and future work. The full-system operation described in this manual uses the ViperX/VX300S arm path.

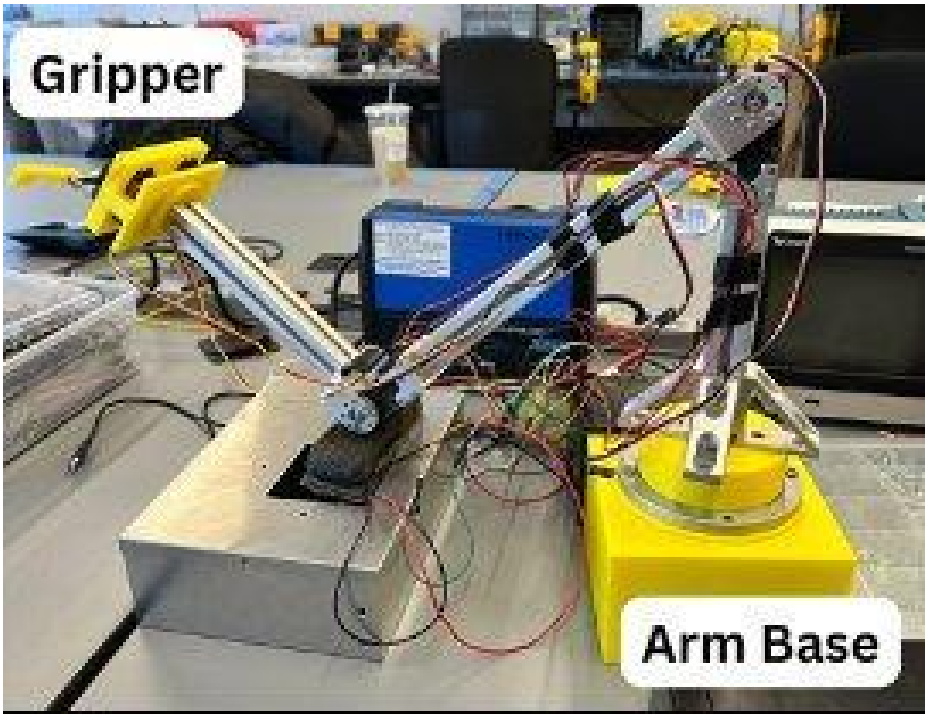


Figure 8. Custom robotic-arm prototype retained from the team draft.

3. Product Setup and Installation

Author source: Darren for hardware assembly; Lion (Xingjian Jiang) for software installation, runtime bring-up, mapping, readiness checks, and integration boundaries. This section explains what another team, maintainer, or store-side operator needs in order to set up GOFR in a new location. After completing these steps, the robot should be ready for the operating procedures described in the User's Manual.

3.1 Start Here: Bring-Up Map

Step	Stage	Exit condition
1	Place and inspect	Place GOFR in a clear indoor test area. Check frame, wiring, batteries, emergency stop, ViperX arm, RealSense camera, PicoScan LiDAR, and ultrasonic sensors.
2	Connect hardware	Connect robot computer, STM32 base controller, ESP32 ultrasonic boards, ViperX arm controller, RealSense camera, and PicoScan LiDAR.
3	Open JetKVM	From a maintenance laptop on the robot network, open http://192.168.8.113 and control the JetPack / robot-host desktop.
4	Verify firmware	Flash or verify STM32 base-control firmware and ESP32 ultrasonic firmware.
5	Install host software	Install Linux dependencies, ROS 2 Humble, Node.js, npm, PostgreSQL, Interbotix/ViperX dependencies, RealSense dependencies, and build tools.
6	Build workspaces	Build the Interbotix workspace first, then the GOFR ROS 2 workspace.
7	Provision backend	Create PostgreSQL database, load schema/seed data, configure backend credentials, and optionally add a Gemini API key.
8	Create or load map	Create the base navigation map for the current room/store or verify the existing map matches the physical layout.
9	Align semantic layout	Use Fleet Manager Store Map to align the home anchor, rack anchors, racks, and product slots.
10	Start runtime stack	Start backend, Fleet Manager UI, navigation, optional rosbridge, ViperX arm stack, camera vision node, and Behavior Tree executor.
11	Run readiness checks	Verify actions, services, topics, UI, semantic map, remote power switches, and E-stop before creating a pickup order.
12	Run first test order	Submit one item such as Apple, Green Tea, Water, Orange, Lemon, Can, or Bag of Chips.

Bring-up rule: There is not currently a single full-system launch command that starts every service. The staged sequence above is the reliable installation path because each subsystem can be verified before the next one is started.

3.2 What's in the Box: System Components

Component	Installation purpose

GOFR mobile chassis	Main robot frame, payload platform, and mounting base for electronics and arm.
Mecanum wheels	Omnidirectional mobile-base movement for aisle alignment and lateral adjustment.
Aluminum frame, acrylic plates, PETG printed parts	Robot structure, protective covers, and sensor/electronics mounting.
ViperX/VX300S robotic arm	Primary validated manipulation path for product pickup and basket placement.
Intel RealSense camera	Product detection, depth sensing, and visual input for the pickup pipeline.
SICK PicoScan LiDAR	Mapping, localization, and navigation sensing.
STM32 base controller	Motor-control and odometry/telemetry bridge between ROS and the mecanum drive.
DJI C620 motor controllers and M3508 motors	Drive system for the four-wheel mecanum base.
ESP32 ultrasonic boards and ultrasonic sensors	Local front/left/right/rear collision-alert sensing.
Robot computer / JetPack host	Runs ROS 2, navigation, vision, manipulation, and task-management software.
Router or local network device	Connects robot services, operator UI, JetKVM, LiDAR, and maintenance laptop.
Batteries and power distribution	Separated compute/network and drive/actuation power branches.
Remote emergency stop	Stops unsafe motion by cutting or disabling the drive/actuation branch.
Two infrared remote power switches	Remote enable/disable controls for the control/compute branch and drive/actuation branch.

Power-branch assumption: The installation assumes separated compute/network and drive/actuation branches. This allows compute to remain available for diagnostics after the drive-side emergency stop has been pressed.

3.3 Hardware Contents from Team Draft

Category	Items
8020 parts	Partially completed first layer; partially completed second layer; 4x 8020-4140; 2x 10.125 in 8020-10; 2x 10 in 8020-10; 4x M3508 holder front; 4x M3508 holder back; 2x 8020-4136; 4x 8020-4151.
Electrical components	4x DJI M3508 motors; 1x Jetson Nano / robot computer; 1x PicoScan LiDAR and mount; 1x JetKVM; 1x Orange RC XT60; 1x JetKVM switch extension port; 1x DJI devboard; 4x batteries; 4x C620 DC controllers; 1x M3508 terminal board; 1x GL.iNet AX1800 wireless router; 1x ViperX 300S robotic arm; 4x ESP32; 6x ultrasonic sensors.
3D-printed parts	2x battery holders; 1x Clustercase; 1x motor driver holder; 1x router holder; 2x charging port; 1x front casing; right/left/front/back/top acrylic sliders; back/front/right/left ultrasonic sensor holders.
Casing and plates	1x 0.125 in x 14 in x 14 in aluminum plate; 1x 0.125 in x 14 in x 16 in aluminum plate; 2x 0.125 in x 10.32 in x 9 in white acrylic plate; 1x 0.125 in x 9 in x 18.3 in white acrylic plate; 1x 0.125 in x 9.5 in x 12.25 in white acrylic plate; 2x 0.125 in x 9.3 in x 3.1 in white acrylic plate.

3.4 Software System Components

Software component	Location	Purpose
Backend API	order-api-postgre	Orders, inventory, users, product data, and semantic map data.
Fleet Manager UI	order-api-postgre/fleet-manager	Operator dashboard, inventory/map views, and Store Map editor.

Navigation package	workspace/src/robot_navigation	LiDAR, Cartographer, Nav2, semantic map, STM32 bridge, and helper CLI.
Task manager package	workspace/src/robot_task_manager	Behavior Tree order execution and full pickup orchestration.
Manipulation package	workspace/src/robot_manipulation	ViperX/VX300S MoveIt stack and /pick_viperx action server.
Vision package	workspace/src/robot_vision	RealSense/YOLO detection and /detections_json topic.
Perception package	workspace/src/robot_perception	Ultrasonic alert topics and local collision sensing integration.
Shared ROS interfaces	workspace/src/robot_interfaces	Custom ROS messages, services, and actions.
ESP32 firmware	ESP32	Ultrasonic and actuator-side firmware projects.
STM32 firmware	STM32	Mobile-base firmware.
Map assets	Maps	Saved navigation maps such as .pbstream, .yaml, and .pgm files.
Semantic map	workspace/src/robot_navigation/config/semantic_map_testmapMain.yaml	Home anchor, approach anchors, racks, slots, navigation poses, and service poses.

3.5 Supporting Equipment and Environment

- Indoor floor area with enough clearance for base motion and arm motion.
- Table, shelf fixture, or test rack for first pickup tests.
- Linux robot host capable of running ROS 2 Humble.
- Maintenance laptop connected to the same robot network.
- JetKVM remote console access to the JetPack / robot host.
- Stable Wi-Fi or Ethernet between the robot, operator computer, JetKVM, and LiDAR network.
- USB access to the STM32, ViperX arm controller, ESP32 boards, and RealSense camera.
- STM32CubeIDE or a compatible STM32 flashing tool.
- ESP-IDF for ESP32 firmware projects and Arduino IDE with ESP32 board support for sketch-level tests.
- PostgreSQL installed locally on the robot/backend host, or Docker Desktop if using the Windows backend test option.
- An operator standing near the emergency stop during all first base-motion and arm-motion tests.

Safety gate: Do not run autonomous navigation or arm pickup until manual stop behavior has been verified. During early tests, the E-stop operator should not also be the person typing commands.

3.6 User and Maintenance Interfaces

Interface	Intended user	Purpose
Customer app	Customer or demo user	Create a shopping/pickup request and view order progress.
Fleet Manager UI	Store employee or operator	Manage inventory/map views, monitor tasks, and prepare store-side operation.
JetKVM remote console	Maintainer or integration operator	Configure JetPack, run terminals, build software, create maps, and run ROS checks.
Infrared remote switches and E-stop controls	Setup operator or safety observer	Enable/disable control and drive/actuation power branches and stop unsafe motion.

3.7 Accounts, Network Settings, and Credentials

Item	Current/default value
Backend URL	http://localhost:3000
Fleet UI URL	http://localhost:5174
Customer app URL	Deployment-specific; record during installation handoff.
JetKVM management URL	http://192.168.8.113
PostgreSQL host	localhost
PostgreSQL port	5432
PostgreSQL database	grocery_inventory
PostgreSQL user	grocerybot
PostgreSQL password	team21
Seed employee login	000AAA / team21
SICK PicoScan LiDAR IP	192.168.8.150
Robot receiver IP	192.168.8.249
Optional rosbridge URL	ws://192.168.8.249:9090

The JetKVM IP is the remote KVM management interface. It is not the ROS host IP and not the LiDAR IP. Use JetKVM to view and control the JetPack / robot-host screen. When commands in this document say localhost, they refer to the JetPack / robot host when those commands or browser checks are run inside the JetKVM-controlled desktop.

Credential warning: Do not commit real API keys. For any real deployment, change the default PostgreSQL password and the default employee password before handing the system to users.

If the Gemini assistant feature is used, create order-api-postgre/config/gemini.json with the following shape:

```
{
  "apiKey": "...",
  "model": "gemini-2.0-flash"
}
```

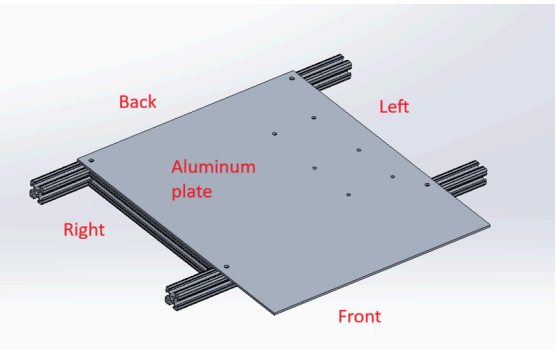
3.8 Physical Setup and Pre-Flight Inspection

1. Place GOFR on a clear, level indoor floor.
2. Make sure the robot cannot roll into people, shelves, loose wires, bags, or fragile objects during first tests.
3. Confirm the ViperX/VX300S arm is mounted securely and has clear space to move.
4. Confirm the RealSense camera is mounted near the ViperX end effector.
5. Confirm the SICK PicoScan LiDAR is mounted at the front of the robot and is not blocked by the frame, battery box, or loose wiring.
6. Confirm ultrasonic sensors are installed for the front, left, right, and rear directions.
7. Confirm the STM32 base controller is connected to the motor controllers.
8. Confirm the ESP32 ultrasonic boards are connected through the expected I2C bus.
9. Confirm the drive/actuation power branch and compute/network power branch are separated.
10. Confirm the two infrared remote switches can enable and disable the control/compute branch and the drive/actuation branch.
11. Confirm the emergency stop can stop unsafe motion.
12. Confirm all batteries are charged and mechanically secured.
13. Confirm XT60/XT30 connectors are fully seated and that no connector is partially mated.

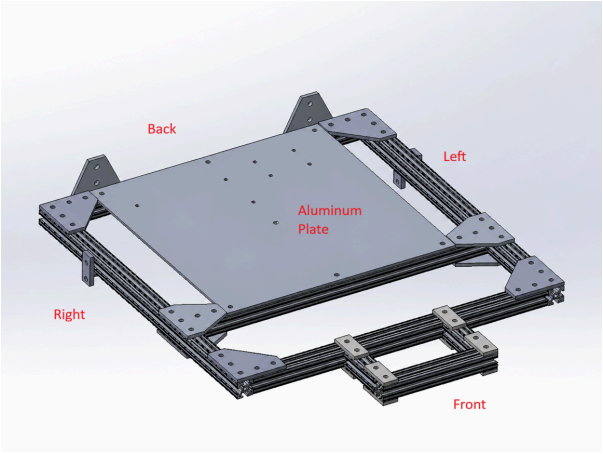
Electrical note: Typical ultrasonic ECHO lines are 5 V, while ESP32 GPIO is 3.3 V only. Use a level shifter or voltage divider before routing an ECHO signal into an ESP32 input pin.

3.9 Chassis Hardware Assembly Procedure

Author source: Darren. The following instructions preserve the existing teammate hardware installation content while clarifying the sequence for a future maintainer.



First-layer plate orientation

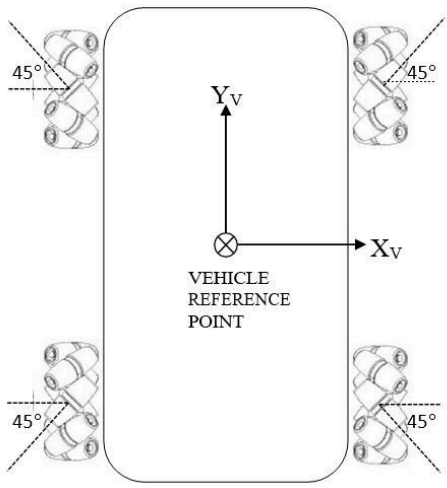


First-layer frame with brackets

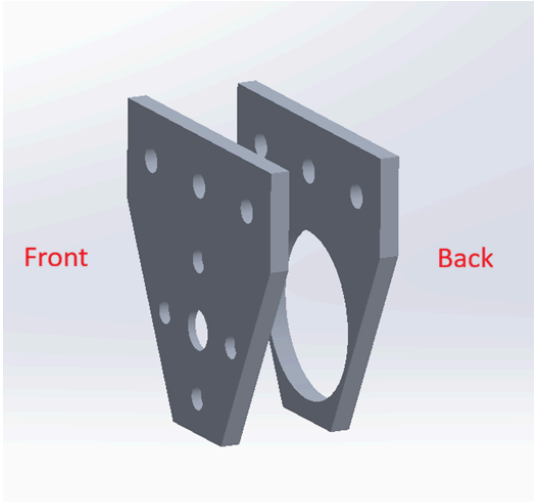
Figure 9. First-layer aluminum plate orientation and assembled lower-frame reference from the original mechanical draft.

First Layer

A partially completed first layer should be provided. Attach the M3508 holders, front and back, into the corners of the first layer using 8-32 screws and nuts placed into the 8020 slots. The front holder should face outward from the first layer. Place each M3508 motor into its holder, attach the flange to the motor with M2 screws, and attach the flange to the mecanum wheel. Make sure the mecanum wheel orientation matches the intended X-configuration so that the force vectors support omnidirectional motion. Repeat this process at all four corners.



Mecanum wheel orientation

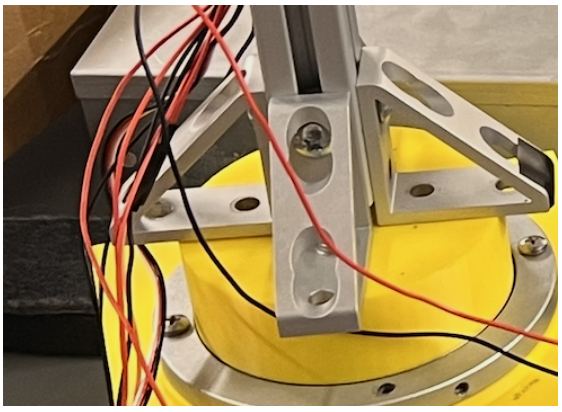


M3508 holder front/back CAD

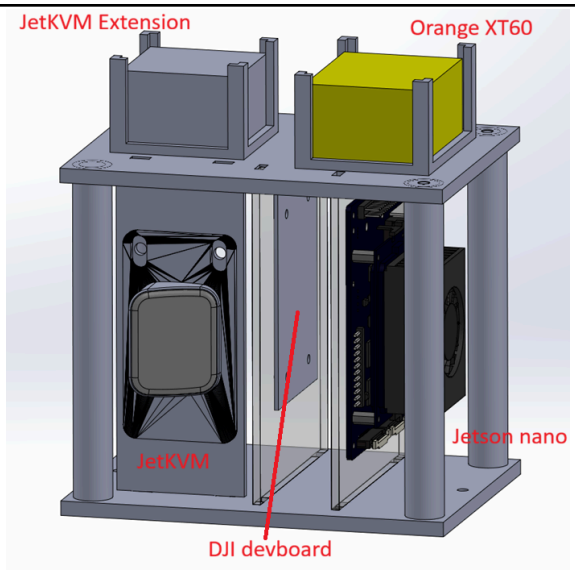
Figure 10. Mecanum wheel X-configuration and M3508 motor-holder front/back reference for the four drive corners.

Place the battery holder in the front corner of the aluminum plate. The bottom of the battery holder has a hole that aligns with the bolt fixing the aluminum plate into the 8020. Place an 8020-4140 vertically and align the holes in the 4140 with the holes in the battery holder. Use 10-32 screws to fix the 4140 into the 8020 bar and battery holder. Repeat this on the left side of the aluminum plate, then attach the batteries into both holder slots.

To attach the motor driver holder, first place the four C620 controllers into the holder, with two on each side of the wall. Fix them using M2 screws and nuts. Attach the M3508 terminal board to the top of the case using M3 screws. Align the two bottom holes of the case with the two center holes in the aluminum plate, then use 10-32 screws and nuts to fix the holder in place.



Battery holder / vertical-bar attachment

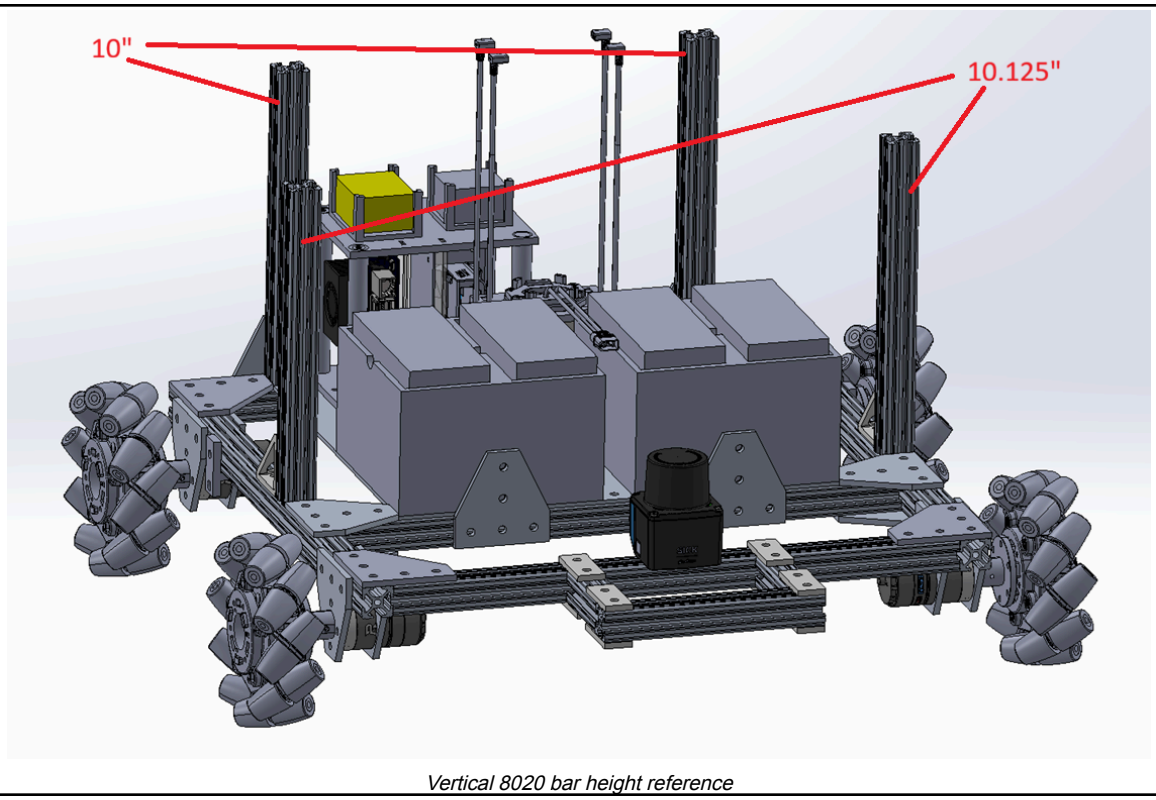


Cluster case electronics layout

Figure 11. Battery-holder/vertical-bar attachment and cluster-case layout for JetKVM, XT60, DJI devboard, and robot computer.

The Clustercase includes a top, base, and three vertical plates. The JetKVM extension port and Orange XT60 slide into the top slots. The smallest plate holds the JetKVM with M3 screws. The second plate holds the DJI devboard with M3 screws. The final plate contains markings for attaching the Jetson Nano / robot computer. Align all vertical plate slots with the base and top parts, then attach the base to the back-right side of the aluminum plate using 10-32 screws and nuts.

Place the GL.iNet router into its holder and mount it to the back-left side of the plate. Place the LiDAR and mounting bracket at the front 8020, centered between the left and right sides of the robot. Vertically place the two 10 inch 8020-10 bars and the two 10.125 inch 8020-10 bars on the robot. The slightly longer 10.125 inch bars should be at the front side. Fix the 10.125 inch front bars with 8020-4136 and fix the two rear bars using 8020-4140. For the front vertical bars, place the front slider set into the bars. Place the right and left slider sets into both the back and front bars, and place the back sliders into the two rear bars.

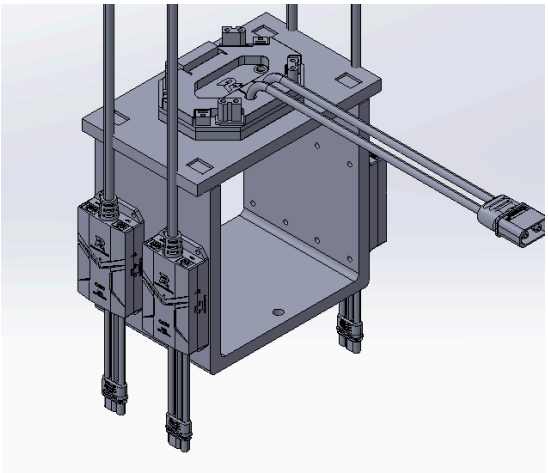


Vertical 8020 bar height reference

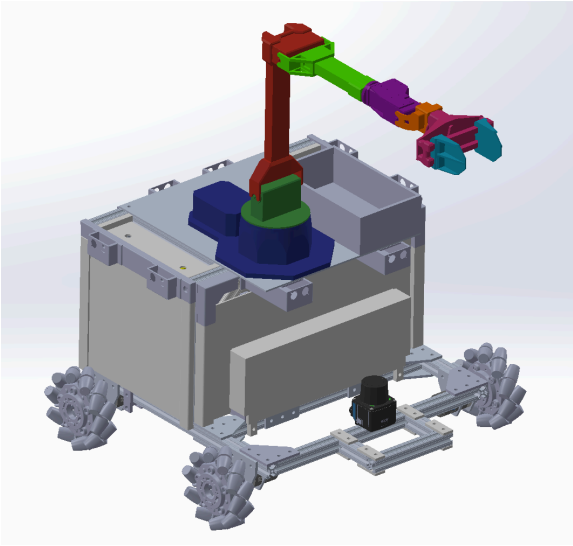
Figure 12. Vertical 8020 bar placement and front/back height reference for joining the first and second chassis layers.

Second Layer

Attach the second layer to the first layer using 8020-2565 on each of the four vertical 8020 bars. Slide the 2565 into the front and back 8020 bars of the second layer, then use 8020-4151 on all four sides of the second layer. Attach four ESP32 holders to the 8020-2565. Attach the ViperX robotic arm to the top plate on the right side and place the basket on the left side. Slide four top sliders into the right and left sides of the 8020 bar.



Second-layer structural CAD

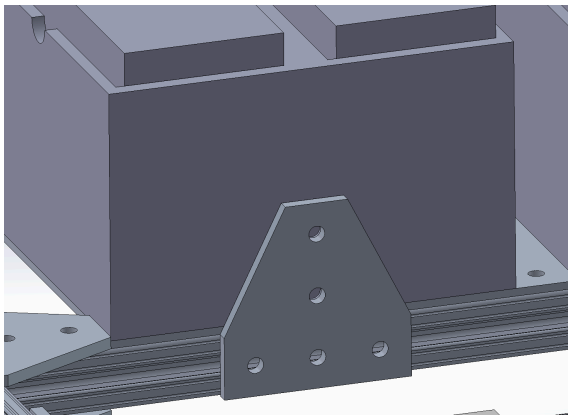


	Arm and basket placement on upper deck

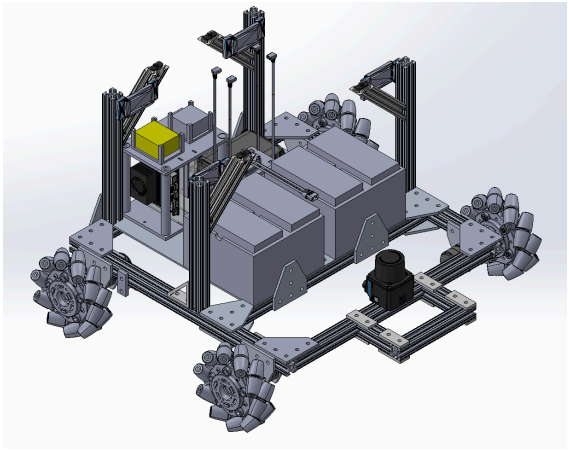
Figure 13. Second-layer structure and top-deck reference for arm and basket placement.

Outer Casing

Slide the 0.125 inch x 9 inch x 18.3 inch white acrylic plate into the front sliders. Place the front casing in front of the battery holders and fix it into the 8020 bar. Slide the 0.125 inch x 10.32 inch x 9 inch white acrylic plate into the right sliders. Slide the 0.125 inch x 9.3 inch x 3.1 inch white acrylic plate into the top-right sliders. Slide the right ultrasonic sensor holder into both acrylic plates and fix it into the 8020 bar using a 10-32 screw. Repeat the corresponding steps for the left side of the robot. Slide the 0.125 inch x 9.5 inch x 12.25 inch white acrylic plate into the back sliders.



Acrylic/front casing detail

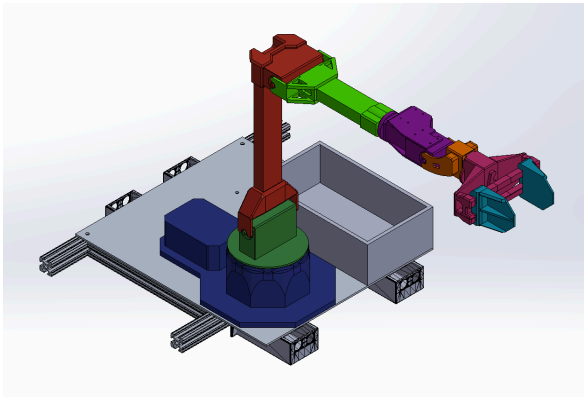


Full chassis CAD with outer structure

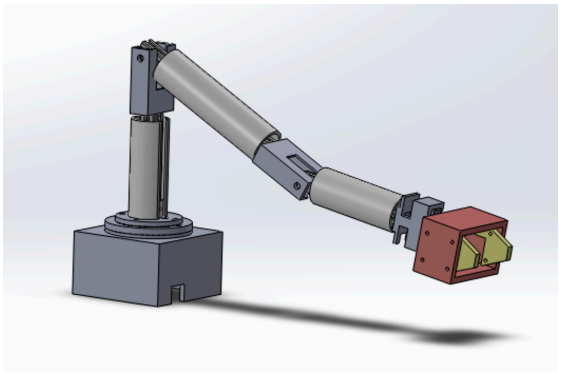
Figure 14. Outer-casing reference and full chassis CAD view for final mechanical closure.

3.10 Custom Robotic Arm Prototype Assembly

Author source: Bach. This section is retained for the prototype custom-arm path. It is not the default ViperX/VX300S installation path.



Custom arm mounted CAD



Custom arm standalone CAD

Figure 15. Custom robotic-arm prototype CAD references retained for the non-default prototype path.

Category	Items
8020 parts	4x 8020 inside-corner brackets; 1x 15 cm 8020 bar; 1x 20 cm 8020 bar; 1x 25 cm 8020 bar.

Electronic components	2x 160 kg*cm servo; 1x 80 kg*cm servo; 1x 60 kg*cm servo; 1x 30 kg*cm servo; 1x ESP32; 1x voltage reducer.
3D printed components	1x robotic arm base; 1x gripper.

14. Attach four 8020 inside-corner brackets to one end of the 15 cm 8020 linkage using sliders and screws. Attach a 160 kg*cm servo to the other end using an M4 screw. Connect the first linkage to the base using the four bottom inside-corner bracket connections with four M4 screws.
15. Attach an 80 kg*cm servo to one end of the 25 cm linkage using an M4 screw. Connect the other end of this linkage to the motor end of the first linkage using the 160 kg*cm motor aluminum bracket.
16. Attach a 60 kg*cm servo to one end of the 20 cm 8020 linkage. Connect the other end of this linkage to the second linkage through the 80 kg*cm motor bracket. Connect the pre-assembled gripper to this linkage using the 60 kg*cm motor bracket.
17. Set up a circuit using a breadboard and 12 V supply. Connect the ESP32 to a personal computer. Connect all servo control signal wires to the ESP32 GPIO. Connect all servo grounds to the 12 V supply circuit. Connect the two 160 kg*cm servo power cables directly to 12 V. Connect the 30, 60, and 80 kg*cm servos to the voltage reducer for 6 V operation. Connect the voltage reducer power and ground to the 12 V circuit.

3.11 Embedded Firmware Setup

STM32 Base Controller

The STM32 controls the mecanum motors, receives velocity commands from ROS, and returns odometry/telemetry. Open the STM32 project and build/flash it with STM32CubeIDE or a compatible STM32 toolchain:

```
STM32/Base_Control_v2/robowalker2024bottominfantry-main/test/test.ioc
```

Parameter	Current value
MCU	STM32F427IIH6
Serial interface	USART2 at 115200 baud
CAN interface	CAN1 at 1 Mbps
Motor IDs	0x201 through 0x204
Command frame	0x200
Drive stack	4x C620 + 4x M3508
Wheel radius	0.076 m
Wheelbase	0.36 m
Wheeltrack	0.30 m

After flashing, connect the STM32 serial device to the robot host. The preferred Linux by-id path for the current full system is:

```
/dev/serial/by-id/usb-Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001-if00-port0
```

ESP32 Ultrasonic and Actuator Boards

Folder	Purpose
ESP32/ultrasonic_I2C	Ultrasonic sensor firmware used by the ROS ultrasonic collision-alert stack.
ESP32/arm_servo_I2C	Custom servo-arm I2C prototype; not the default ViperX path.
ESP32/linear_actuator	Linear-actuator prototype.
ESP32/ultrasonic	Arduino ultrasonic sketch.
ESP32/servo-component-testing	Arduino servo test sketches.

Typical ESP-IDF flashing flow:

```
cd ESP32/ultrasonic_I2C
idf.py set-target esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

Direction	I2C address
Front	0x09
Left	0x10
Right	0x11
Back	0x12

Signal	Pin
Channel 1 TRIG	GPIO4
Channel 1 ECHO	GPIO16
Channel 2 TRIG	GPIO17
Channel 2 ECHO	GPIO34
I2C SDA	GPIO21
I2C SCL	GPIO22

3.12 Robot Host Software Setup

18. Connect the operator computer to the robot network.
19. Open http://192.168.8.113 in a browser.
20. Log in to the JetKVM system.
21. Confirm that the JetPack / robot-host display is visible.
22. Open a terminal on the JetPack desktop.
23. Run the remaining Linux, ROS, backend, mapping, and readiness-check commands from that terminal unless a step explicitly says to use another machine.

Install these prerequisites on the Linux robot host: ROS 2 Humble, Python 3, colcon, Node.js 20+, npm 10+, PostgreSQL 14+, Interbotix dependencies for the ViperX/VX300S arm, Intel RealSense SDK and Python dependencies, rosbridge_server if remote map viewing is needed, and deep-translator if inventory translation is used.

Build Interbotix Workspace

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws
source /opt/ros/humble/setup.bash
colcon build
```

Build GOFR ROS 2 Workspace

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
colcon build --base-paths src
source install/setup.bash
```

Source Every ROS Terminal

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
```

3.13 Backend and Fleet UI Setup

Linux Backend Setup

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
```

Create the database role and database:

```
sudo -u postgres psql
```

Inside psql, run:

```
CREATE ROLE grocerybot WITH LOGIN PASSWORD 'team21';
CREATE DATABASE grocery_inventory OWNER grocerybot;
\q
```

Load schema and seed data:

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre
PGPASSWORD=team21 psql -h localhost -U grocerybot -d grocery_inventory -f db/init/001_schema.sql
PGPASSWORD=team21 psql -h localhost -U grocerybot -d grocery_inventory -f db/init/002_seed.sql
```

Install backend dependencies and start the server:

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre
npm install
python3 -m pip install deep-translator
export PGHOST=localhost
export PGPORT=5432
export PGDATABASE=grocery_inventory
export PGUSER=grocerybot
export PGPASSWORD=team21
npm run dev
```

Expected backend URL: http://localhost:3000

Fleet Manager UI

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre/fleet-manager
npm install
npm run dev -- --host 0.0.0.0
```

Open the UI and log in using the seeded account:

```
http://localhost:5174


000AAA / team21
```

Windows Backend Option

For a Windows-only backend/UI test, use Docker Desktop for PostgreSQL. This path is useful for backend/UI testing, but the robot ROS stack is intended to run on the Linux robot host.

```
cd F:\EC463\EC463_Team_21_Grocery_Robot\order-api-postgre
docker compose up -d
npm install
npm run dev


cd F:\EC463\EC463_Team_21_Grocery_Robot\order-api-postgre\fleet-manager
npm install
npm run dev
```

3.14 Store Map and Semantic Layout Setup

This setup is required before the frontend order workflow can drive the robot automatically. The app needs both a base navigation map and a semantic store layout so a product order such as Apple can become a navigation goal and pickup target.

Create the Base Navigation Map

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
ros2 run robot_navigation nav_assistant mapping-stack
```

Drive the robot manually through the store aisle or test area:

```
ros2 run robot_navigation nav_assistant teleop
```

Save and export the map:

```
ros2 run robot_navigation nav_assistant save-map --map-name testmapMain
ros2 run robot_navigation nav_assistant export-map --map-name testmapMain
```

This should produce Maps/testmapMain.pbstream, Maps/testmapMain.yaml, and Maps/testmapMain.pgm. For an existing demo setup, verify that these files match the physical store layout before continuing.

Open the Store Map Editor

24. Start the backend and Fleet Manager UI if they are not already running.
25. Open http://localhost:5174.
26. Log in with 000AAA / team21.
27. Open the Store Map page. It loads the base map image and the current semantic map.

Align the Semantic Store Layout

28. Confirm the base map is the correct map for the current room or store aisle.
29. Confirm the map origin and map orientation match the physical setup. If the base map is wrong, rebuild and export the map before editing semantic objects.
30. Enable Edit Mode.
31. Position the Home anchor at the robot's normal start/return location.
32. Position approach anchors near the shelf areas the robot should serve.
33. Drag each rack rectangle to the physical rack or shelf location.
34. Rotate racks or anchors when their yaw does not match the real shelf direction.
35. Position each product slot at the service location for that product.
36. Use the manual pose editor for precise x, y, z, and yaw values when dragging is not accurate enough.
37. Check that each product slot is associated with the correct product name and product ID.

Object type	Meaning
Anchor	Named pose such as Home or a rack approach point.
Rack	Physical shelf/rack footprint and orientation.
Slot	Product-specific target with nav_pose, service_pose, rack level, product ID, and product name.

Products are resolved through slots. The current semantic map includes product slots such as Green Tea, Water, Apple, Orange, Lemon, Can, and Bag of Chips.

Save and Verify the Semantic Map

Click Save in the Store Map page. The system saves the active semantic map and records version history. The active semantic file is:

```
workspace/src/robot_navigation/config/semantic_map_testmapMain.yaml
```

- The Store Map page shows the expected anchors, racks, and slots.
- The selected object panel shows reasonable x, y, z, and yaw values.
- Product names in slots match the names used in test orders.
- The backend can still load the Store Map page after refresh.
- The ROS semantic map service is available after the navigation stack starts.

Semantic-map dependency: If the semantic map is missing or wrong, the frontend may still create an order, but the robot may navigate to the wrong place or fail semantic resolution.

3.15 Full System Startup

Start the following processes in separate terminals after the base map and semantic layout have been created or verified. Source the ROS environment in every ROS terminal.

Terminal 1: Backend

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre
npm run dev
```

Terminal 2: Fleet Manager UI

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre/fleet-manager
npm run dev -- --host 0.0.0.0
```

Terminal 3: Navigation Stack

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
ros2 run robot_navigation nav_assistant localization-stack \
  --map-name testmapMain \
  --with-nav2-rviz false \
  --serial-port
  /dev/serial/by-id/usb-Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001-if00-port0
```

This starts localization, Nav2, semantic map service, and the STM32 serial bridge.

Terminal 4: Optional Remote Map Bridge

```
sudo apt install ros-$ROS_DISTRO-rosbridge-server

cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

The embedded app should connect to ws://192.168.8.249:9090.

Terminal 5: ViperX Arm and /pick_viperx

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
ros2 launch robot_manipulation vx300_moveit.launch.py \
```

```
  robot_model:=vx300s \
  robot_name:=vx300s \
  motor_port:=/dev/serial/by-id/usb-FTDI_USB__-__Serial_Converter_FT88YTG6-if00-port0
```

This provides MoveIt, ViperX controllers, and the /pick_viperx action server.

Terminal 6: Camera Vision

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
ros2 run robot_vision camera_vision --ros-args \
  -p parent_frame:=vx300s/ee_gripper_link \
  -p camera_mount_frame:=camera_mount_frame \
  -p camera_optical_frame:=camera_color_optical_frame
```

For a live OpenCV preview, add:

```
-p show_live_window:=true
```

Camera ownership: Do not launch realsense2_camera separately for the current ViperX pickup flow. The camera_vision node owns the RealSense pipeline and publishes the camera TF frames used by the Behavior Tree and arm stack.

Terminal 7: Behavior Tree Executor

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash
ros2 run robot_task_manager bt_executor_viperX
```

Launch conflict warning: Do not launch robot_manipulation vx300_auto_pick.launch.py for the Behavior Tree full-system flow. That older launch starts vision_auto_pick.py, which can compete with bt_executor_viperX for /detections_json and /pick_viperx. The capital X in bt_executor_viperX matters.

3.16 Readiness Checks

Before giving the robot to a user, run these checks from the JetPack / robot-host terminal through JetKVM unless noted otherwise. If any check fails, stop and troubleshoot before running a user-facing pickup task.

System	Command or check	Expected result
Remote console	Open http://192.168.8.113 from the operator computer	JetKVM shows the JetPack / robot-host screen.
Backend	Open http://localhost:3000	Backend is listening.
UI	Open http://localhost:5174	Login page or dashboard appears.
Customer app	Open deployed customer app URL	Order page appears.
Arm action	ros2 action list | grep pick_viperx	/pick_viperx
Navigation action	ros2 action list | grep navigate_to_pose	/navigate_to_pose
Semantic map	ros2 service list | grep semantic_map	/semantic_map/resolve_target
Camera detections	ros2 topic hz /detections_json	Topic publishes when camera is active.
One detection payload	ros2 topic echo /detections_json --once	JSON detection payload appears when target is visible.
Order service	ros2 service list | grep order	/order/new
Semantic layout	Refresh Store Map page	Correct home anchor, racks, and product slots.
Remote power controls	Test both infrared remote switches during setup	Control/compute and drive/actuation branches respond as expected.
Emergency stop	Press E-stop during a controlled test	Unsafe motion stops.
Optional rosbridge	ros2 node list | grep rosbridge	rosbridge node is present if remote map viewing is needed.

3.17 First Test Order

Use a single item for the first test. Recommended item names already used by the semantic map include Green Tea, Water, Apple, Orange, Lemon, Can, and Bag of Chips.

For a direct ROS-side integration test, call:

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/install/setup.bash


cat > /tmp/apple_order.yaml <<'EOF'
order:
  order_id: 1
  role: customer
  requester_id: test
  items:
    - product_id: '3'
      name: Apple
      aisle: '0'
      rack: 0
      shelf_level: 2
      qty: 1
      price: 0.0
      stock: 1
EOF


REQUEST="$(cat /tmp/apple_order.yaml)"
ros2 service call /order/new robot_interfaces/srv/NewOrder "$REQUEST"
```

Quantity constraint: Use qty: 1 for current pickup testing. The current implementation handles one physical pickup per order line; larger quantities can create a mismatch between physical pickup count and inventory decrement.

3.18 Expected Result of a Full Pickup Run

38. The backend stores the order.
39. bt_executor_viperX polls or receives the order.
40. The semantic map resolves the navigation target for the item.
41. Nav2 drives the base to the shelf stop pose.
42. The arm moves to a scan pose.
43. camera_vision publishes detections on /detections_json.
44. The Behavior Tree filters the detection and generates pregrasp, grasp, lift, and place poses.
45. /pick_viperx executes the grasp through MoveIt.
46. The item is placed into the next basket pose.
47. Inventory is updated through the backend.
48. The order is marked done or failed.

Current basket placement is sequential. The first picked object goes to basket slot 1, the second to basket slot 2, and so on. Basket placement is not yet type-aware.

3.19 Software Updates After a New Release

49. Pull or copy the new repository version onto the robot host.
50. Read release notes or team handoff notes for changed packages.
51. Rebuild affected ROS packages. For full integration changes, use the command below.

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace
source /opt/ros/humble/setup.bash
source /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/workspace/interbotix_ws/install/setup.bash
colcon build --base-paths src --packages-select \
  robot_interfaces robot_navigation robot_manipulation robot_vision robot_task_manager
source install/setup.bash
```

If backend dependencies changed:

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre
npm install
```

If Fleet UI dependencies changed:

```
cd /home/grocerybot/Desktop/EC463_Team_21_Grocery_Robot/order-api-postgre/fleet-manager
npm install
```

- If database schema changed, back up the database before running migrations or reloading seed data.
- If embedded firmware changed, reflash the matching STM32 or ESP32 target before running the full stack.
- Repeat all readiness checks before autonomous or user-facing operation.

3.20 Known Installation Boundaries

- The ViperX/VX300S path is the current primary validated path.
- The custom servo arm and rack path is not the default full-system deployment path.
- The robot status page in the UI may contain placeholder behavior and should not be treated as complete live telemetry unless updated.
- The SLAM web page requires rosbridge_server and the correct robot host/IP.
- Shelf-level semantic poses are resolved in the current customer pickup path, but shelf-level arm pose execution is not fully connected in the current customer tree.
- The current customer pickup flow is fail-fast. It does not yet support per-item skip, partial-success settlement, or continue-after-failure behavior.
- Return-home recovery has a retry budget, but a recovered return home still preserves the order result as failed if the original order failed.

3.21 Setup Completion Checklist

- [ ] The robot powers on without loose wiring, battery, or connector issues.
- [ ] The two infrared remote switches can control the control/compute and drive/actuation power branches.
- [ ] The emergency stop can stop unsafe motion.
- [ ] JetKVM at http://192.168.8.113 can access the JetPack / robot-host desktop.
- [ ] The backend and UI are reachable.
- [ ] The customer app and Fleet Manager are reachable for daily users.
- [ ] The base map has been created or verified for the current location.
- [ ] The Store Map semantic layout has been saved with the correct home anchor, racks, and product slots.
- [ ] The navigation action is available.
- [ ] The semantic map service is available.
- [ ] The arm action server is available.
- [ ] The camera publishes detections.
- [ ] A one-item test order can be accepted by the Behavior Tree.

- [ ] The operator knows how to stop the robot and who to contact for maintenance.

4. User's Manual

Author source: Lion (Xingjian Jiang). This section describes how to use and operate GOFR after the setup and installation process has been completed. It assumes the base map, semantic store layout, backend, Fleet Manager UI, customer app, robot host, navigation stack, camera stack, arm stack, and Behavior Tree executor have already been brought up by a maintainer. It also assumes that the installer will walk the client through these procedures during customer installation.

4.1 User Roles

Role	Main interface	Main responsibility
Customer or demo user	Customer app	Submit a pickup request and monitor order progress.
Store employee / operator	Fleet Manager UI and physical safety controls	Monitor robot readiness, store map, inventory, and task progress.
Maintainer / integration operator	JetKVM remote console, Fleet Manager, physical controls	Recover from abnormal states, restart services, and run diagnostic checks.

Daily users should use the customer app and Fleet Manager UI. Command-line access is reserved for setup, maintenance, and recovery.

4.2 Before Operation

Before starting a pickup run, the store employee or maintainer should confirm:

- The robot is in the mapped store/test area.
- The aisle and arm workspace are clear of people, loose cables, bags, and fragile objects.
- The two infrared remote power switches are available and working.
- The emergency stop is reachable.
- The customer app is reachable.
- The Fleet Manager UI is reachable.
- The Store Map page shows the correct home anchor, racks, and product slots.
- The robot has already passed the setup readiness checks.

Do not start a pickup task: Do not start a pickup task if the map is wrong, the robot is outside the mapped area, the camera is blocked, or the emergency stop is not reachable.

4.3 Operating Mode 1: Customer Pickup

Purpose

Customer Pickup mode lets a customer or demo user request grocery items from the customer app. The robot uses the saved semantic store map to navigate to the product area, detect the target object, pick it with the ViperX arm, place it in the basket area, and update order progress.

User Interface

The customer uses the customer app. The exact deployment URL is deployment-specific and should be recorded during installation handoff. The customer app should provide product browsing or item selection, order submission, and order status/progress display.

Normal Operation

52. Open the customer app.
53. Select the desired product.
54. For the current prototype, use one item per order line and set quantity to 1.
55. Submit the order.
56. Wait for the app or operator to show that the order has been received.
57. Keep clear of the robot path and arm workspace while the robot is moving.
58. When the robot finishes, retrieve the item only after the robot has stopped moving.

The expected robot response is: backend records the order; task manager accepts it; semantic map resolves the product into a navigation target; mobile base navigates to the product area; ViperX moves to a scan pose; RealSense and vision stack search for the object; the arm picks the item and places it into the basket; backend updates inventory and order state; robot returns home after success or attempts failure recovery if the order fails before completion.

Abnormal Results and Recovery

Abnormal result	What the user may see	Recovery
Product name does not match the semantic map	Order does not execute or fails quickly	Store employee checks product name and slot assignment in Fleet Manager.
Robot cannot navigate to the target	Robot stops or reports failure	Store employee clears obstacles; maintainer checks navigation and semantic map.
Camera cannot detect the item	Robot scans but does not pick	Store employee checks object visibility and lighting; maintainer checks /detections_json.
Arm cannot pick the object	Robot may report failed order	Store employee keeps area clear; maintainer runs arm/camera diagnostics.
Quantity is greater than 1	Inventory may not match physical pickup count	Use qty: 1 per order line until this feature is updated.
Robot returns home after failure	Order remains failed even if robot reaches home	This is expected failure-recovery behavior.

Exit or Stop

The customer should not physically stop the robot unless there is immediate danger. If the robot is operating unsafely, press the emergency stop or ask the store employee to press it, keep clear of the robot, and wait for the store employee or maintainer to recover the system.

This feature not yet implemented: Customer-side order cancellation from the app is not documented as a reliable active stop mechanism for the current robot. Use the store employee safety procedure for stopping active robot motion.

4.4 Operating Mode 2: Store Employee / Fleet Manager Operation

Purpose

Store Employee mode is used by a store employee or demo operator to monitor robot readiness, view inventory, inspect the semantic store map, and observe task progress. This role should not require command-line access during normal operation.

User Interface

Open the Fleet Manager UI at http://localhost:5174 when viewing from the JetKVM-controlled robot host. If viewing from another laptop, use the deployed network address for the Fleet Manager server. Log in with 000AAA / team21 for first bring-up, and replace the password for real deployment.

Normal Operation

59. Open Fleet Manager.
60. Log in as an employee.
61. Open the dashboard or robot status page.
62. Confirm the robot is ready for a task.
63. Open the Store Map page.
64. Confirm the home anchor, racks, and product slots match the physical store layout.
65. Open the inventory page and confirm the target product is present.
66. Watch the customer app or backend order status while the robot executes the order.
67. Keep the aisle clear and monitor the emergency stop.

The expected system response is that Fleet Manager loads inventory and semantic map data from the backend, Store Map displays anchors/racks/slots on top of the saved map, backend receives and tracks orders, and robot_task_manager consumes pending orders when the ROS stack is running.

Store Map Viewing

Use the Store Map page to verify that the semantic layout is still correct. During normal operation, do not move anchors, racks, or slots unless the physical store layout has changed. If a rack or product has been moved, pause operation, enable edit mode, move the affected semantic object, save the semantic map, ask the maintainer to verify the semantic map service if needed, and run a one-item test before returning to customer operation.

Inventory Viewing

Use the inventory page to check product names and stock values. Product names should match semantic map product slot names. Current test product names include Green Tea, Water, Apple, Orange, Lemon, Can, and Bag of Chips.

Abnormal Results and Recovery

Abnormal result	Store employee action
Fleet Manager cannot load	Ask maintainer to check backend and UI services.
Store Map is wrong	Stop operation; correct and save semantic map before running pickup.
Product is missing from inventory	Update inventory or use a valid product.
Robot status page does not match physical robot	Treat physical robot and maintainer checks as authoritative.
Robot moves toward an obstacle	Press emergency stop.
Order fails	Keep the area clear; ask maintainer to check backend, navigation, vision, and arm status.

This feature not yet implemented: The robot status page may contain placeholder or incomplete live telemetry depending on the current software version. Do not rely on it as the only safety indicator.

Exit or Stop

To stop normal monitoring, log out of Fleet Manager or close the browser tab. To stop active robot operation, press the emergency stop if there is immediate danger, use the infrared remote switch to disable the relevant power branch when instructed by the maintainer, keep people away from the robot until it is confirmed safe, and do not restart an order until the abnormal state has been cleared.

4.5 Operating Mode 3: Maintainer Diagnostics and Recovery

Purpose

Maintainer mode is used only when the robot must be checked, restarted, recovered, or debugged. This mode uses JetKVM and may use command-line tools. It is not a daily customer or store employee workflow.

User Interface

Use any maintenance laptop connected to the robot network and open http://192.168.8.113. This opens JetKVM, which controls the JetPack / robot-host desktop and terminal. When commands are run from this desktop, localhost refers to the JetPack / robot host.

Diagnostic Checks

System	Check
Backend	Open http://localhost:3000
Fleet Manager	Open http://localhost:5174
Arm action	ros2 action list | grep pick_viperx
Navigation action	ros2 action list | grep navigate_to_pose
Semantic map service	ros2 service list | grep semantic_map
Camera detections	ros2 topic hz /detections_json
Order service	ros2 service list | grep order
Optional remote map bridge	ros2 node list | grep rosbridge

Recovery Procedure

68. Stop unsafe motion with the emergency stop if needed.
69. Keep the aisle and arm workspace clear.
70. Record what mode the robot was in: navigation, scan, pick, basket place, return home, or idle.
71. Check whether the backend and Fleet Manager are still reachable.
72. Check whether /navigate_to_pose, /pick_viperx, /detections_json, and /semantic_map/resolve_target are available.
73. Confirm the semantic map still matches the physical rack/product layout.
74. Confirm the target object is visible to the camera.
75. Reset or restart only the failed subsystem when possible.
76. Run a one-item test order with qty: 1.
77. Return the robot to customer operation only after the test passes.

Built-In Recovery Behavior

The current customer pickup tree has return-home recovery. If a customer order fails before all items are complete, the robot attempts to navigate back to the configured home_goal. The order remains marked as failed even if the robot successfully returns home. This behavior is intentional because a failed pickup should not be reported as successful just because the robot recovered its position.

This feature not yet implemented: GOFR does not currently provide a single in-app self-test button that verifies backend, navigation, arm, camera, semantic map, and safety controls together. For now, maintainers use the readiness checks in this manual as the diagnostic mode.

4.6 Operating Mode 4: Store Map Adjustment

Purpose

Store Map Adjustment mode is used when the physical store layout changes. It should be performed by a store employee with maintainer support or by the maintainer directly.

User Interface

Use Fleet Manager at http://localhost:5174 and open the Store Map page.

Normal Operation

78. Stop active robot tasks.
79. Open Store Map.
80. Confirm the base map still matches the physical space.
81. Enable edit mode.
82. Move the home anchor if the robot start location changed.
83. Move approach anchors if the robot should approach shelves from a different location.
84. Move or rotate racks if shelves moved.
85. Move product slots if product locations changed.
86. Save the semantic map.
87. Run a one-item pickup test before allowing customer use.

The expected system response is that the UI saves the active semantic map, the backend records semantic map version history, and the ROS semantic map server can resolve future orders against the updated semantic map.

Abnormal Results and Recovery

Abnormal result	Recovery
Base map is wrong	Rebuild and export the navigation map before editing semantic objects.
Product slot does not match inventory name	Correct product name or product ID before running pickup.
Robot navigates to the wrong side of a rack	Adjust slot nav_pose, rack yaw, or approach anchor.
Robot stops too far from the product	Ask maintainer to verify semantic pose and Nav2 local costmap behavior.

This feature not yet implemented: Shelf-level semantic service_pose is not fully connected to the current customer pickup arm execution path. The current system uses semantic targets mainly for navigation and product/rack context.

4.7 Operating Mode 5: Emergency Stop and Power Control

Purpose

Emergency Stop and Power Control mode is used to stop unsafe motion or to disable parts of the robot during setup and recovery.

Control	Use
Emergency stop	Stop unsafe robot motion immediately.
Infrared remote switch for control/compute branch	Enable or disable control/compute power during setup or recovery.
Infrared remote switch for drive/actuation branch	Enable or disable drive/actuation power during setup or recovery.

Normal Stop Procedure

88. Wait until the robot is idle.
89. Close or log out of customer app and Fleet Manager sessions.
90. Ask the maintainer to stop running ROS/backend processes.
91. Use the infrared remote switches to disable power branches as appropriate.
92. Confirm the robot is no longer moving.

Emergency Stop Procedure

93. Press the emergency stop.
94. Do not stand in the robot path or arm workspace.
95. Tell the maintainer what happened.
96. Do not release or reset the robot until the maintainer confirms the cause.
97. After recovery, run readiness checks before the next order.

4.8 Operating Limits and Not-Yet-Implemented Features

- Use one physical pickup per order line. Current pickup tests should use qty: 1.
- Basket placement is sequential, not item-type-aware.
- Customer-side cancellation is not documented as a reliable active stop mechanism.
- Partial-success order settlement is not implemented.
- Full shelf-level arm motion driven directly by semantic service_pose is not yet implemented in the customer pickup tree.
- A single in-app self-test button is not implemented.
- Multi-robot fleet operation is future work.
- The robot status page may not contain complete live telemetry in the current software version.

These limits should be reviewed with the client during installation handoff.

4.9 Where to Get Help

98. Stop unsafe motion first.
99. Record the operating mode, item name, and visible error.
100. Check Fleet Manager and the customer app for order state.
101. Ask the maintainer to inspect JetKVM and ROS diagnostics.
102. Use the troubleshooting section of the full documentation for subsystem-specific recovery.

For course or project maintenance, contact the GOFR team member assigned to the affected subsystem.

5. Safety Issues

Author source: Pree for general robot and data/software safety; Darren for water-exposure hazard; Bach for custom-arm startup hazard; Lion for final consolidation.

5.1 General Robot Safety

- GOFR has moving wheels, an articulated robotic arm, batteries, and active sensing hardware.
- Keep operators and bystanders clear of the arm workspace before enabling motion.
- Ensure the remote E-stop is accessible during every run.
- Do not place hands inside the arm or wheel path while the robot is active.
- Do not operate with loose wiring, unstable battery connections, or exposed power terminals.
- Use SLAM and robot-status pages as supervisory tools only; they are not substitutes for direct human safety observation.

5.2 Data and Software Safety

- Employee pages require authentication.
- Database credentials are currently simple development credentials and should be changed for deployment outside a lab environment.
- API keys stored in config/gemini.json must not be committed to public repositories.

5.3 Recovery Guidance for Common Abnormal States

- If the robot behaves unpredictably, activate the physical E-stop first and investigate software second.
- If localization drifts or the robot no longer tracks the map correctly, stop navigation, relaunch localization, and verify the active map files.
- If the arm stalls or cannot complete a pick, stop the arm action, verify the USB connection and MoveIt bring-up, then retest with a preview or diagnostic launch.
- If the UI becomes unresponsive while ROS is still running, do not assume the robot is safe; use the E-stop or terminate motion nodes directly.

5.4 Water and Custom-Arm Hazards

- **Darren:** Keep the robot away from water sources because holes in the casing could allow water to leak into the system.
- **Bach:** Maintain a sufficient distance away from the custom robotic arm during startup because the motors can move quickly while returning to their initial position.

Appendix A: Glossary

The following terms should remain consistent across the installation section, user manual, safety section, and troubleshooting section.

Term	Definition
GOFR	Grocery Operations and Fulfillment Robot; the mobile manipulator developed by Team 21.
ROS 2	Robot Operating System 2; middleware used for robot nodes, topics, services, actions, and launch processes.
Behavior Tree	The task-execution structure used by GOFR to sequence order handling, navigation, perception, picking, placement, and recovery.
Nav2	ROS 2 navigation framework used for localization-to-goal driving and navigation actions.
Cartographer	SLAM system used to build and localize against 2D maps.

Semantic map	A store-aware map layer that connects product names/IDs to anchors, racks, slots, navigation poses, and service poses.
Service pose	A pose near a shelf or product slot where the robot or arm can perform a service action such as scanning or pickup.
ViperX/VX300S	The Interbotix robotic arm used as the primary validated manipulation platform.
MoveIt	Motion planning framework used to command the ViperX arm and execute pick trajectories.
RealSense	Intel depth camera used for visual perception and product detection.
PicoScan LiDAR	SICK 2D LiDAR sensor used for mapping, localization, and navigation.
STM32	Microcontroller used for the mobile-base controller and odometry/telemetry bridge.
ESP32	Microcontroller used for ultrasonic sensor boards and prototype actuator-side firmware.
Emergency stop	Hardware stop control intended to stop unsafe motion by cutting or disabling the drive/actuation branch.
Infrared remote switch	Remote power switch used to enable or disable the compute/control or drive/actuation power branch.
JetKVM	Remote KVM device used to access the JetPack / robot-host desktop and terminal.
JetPack / robot host	The Linux computer that runs ROS, backend services, vision, navigation, and task-management processes.
/pick_viperx	ROS action server used to request ViperX pick execution.
/detections_json	ROS topic published by the camera vision node with object-detection payloads.
/navigate_to_pose	ROS action used by Nav2 to send the mobile base to a navigation goal.
/order/new	ROS service used to submit an order directly to the Behavior Tree task manager.
Fleet Manager	Operator web UI used for inventory/map views, Store Map editing, and task monitoring.
Customer app	Customer-facing web interface used to browse products and submit pickup requests.
PostgreSQL	Relational database used by the backend for inventory, order, user, and map-related data.
rosbridge	WebSocket bridge that allows web clients to communicate with ROS topics/services/actions.